

1. Function with Parameters

Theory:

Functions allow code reusability. By passing parameters, we can send data into a function for processing.

Syntax:

```
return_type function_name(data_type parameter1, data_type parameter2) {  
    // code  
}
```

Example:

```
#include <stdio.h>
```

```
void greet(char name[]) {  
    printf("Hello, %s!\n", name);  
}
```

```
int main() {  
    greet("Alice");  
    return 0;  
}
```

2. Local Variables

Theory:

Local variables are declared inside a function or block and can be used only within it.

Syntax:

```
void function() {  
    int localVar = 10; // local variable  
}
```

Example:

```
#include <stdio.h>
```

```
void display() {  
    int x = 5; // local to display()  
    printf("Local x: %d\n", x);  
}
```

```
int main() {  
    display();  
    return 0;  
}
```

3. Global Variables

Theory:

Global variables are declared outside all functions. They can be accessed and modified by any function in the program.

Syntax:

```
int globalVar = 10;
```

```
void func() {  
    printf("%d", globalVar);  
}
```

Example:

```
#include <stdio.h>
```

```
int count = 0; // global variable
```

```
void increment() {
```

```
    count++;  
}  
  
int main() {  
    increment();  
    increment();  
    printf("Count: %d\n", count); // Output: 2  
    return 0;  
}
```

4. Static Variables

Theory:

Static variables retain their value between function calls. Their lifetime is throughout the program execution.

Syntax:

```
void function() {  
    static int counter = 0;  
    counter++;  
}
```

Example:

```
#include <stdio.h>
```

```
void demo() {  
    static int x = 0;  
    x++;  
    printf("x = %d\n", x);  
}
```

```
int main() {  
    demo();  
}
```

```
demo(),
demo(); // x retains value
return 0;
}
```

5. Recursive Functions

Theory:

A function that calls itself is called a recursive function. It must have a base condition to terminate.

Syntax:

```
int function(int num) {
    if (base_case)
        return result;
    else
        return function(modified_input);
}
```

Example:

```
#include <stdio.h>
```

```
int factorial(int n) {
    if (n == 0)
        return 1; // base case
    else
        return n * factorial(n - 1);
}
```

```
int main() {
    int result = factorial(5);
    printf("Factorial: %d\n", result);
    return 0;
}
```